コンピュータシステム2

第4回 並行プロセスと相互排除・同期

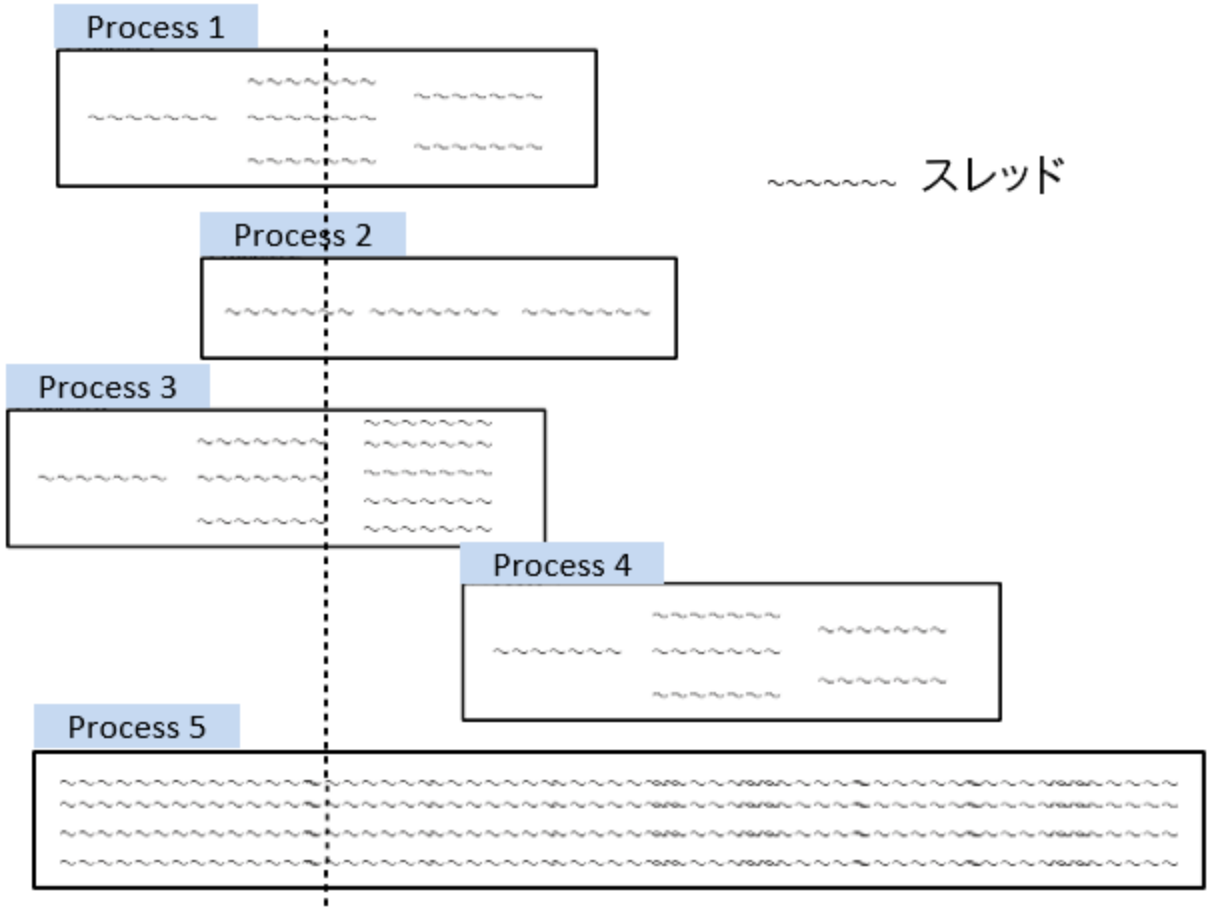
神戸大学 DX・情報統括本部 情報基盤センター
大学院 システム情報学研究科
工学部 情報知能工学科
鳩野逸生・大野麻子

本日の講義内容

- 講義内容
- プロセス関連復習
- 並行プロセスと資源
- 相互排除と同期
  - 必要性
  - 際どい領域
  - 同期と相互排除
- 相互排除機構
  - 割り込み禁止
  - Busy-wait
  - ロック変数
  - 同期基本命令
  - Dekker のアルゴリズム
- Sleep and Wakeup による相互排除・同期
  - セマフォ
  - モニタ

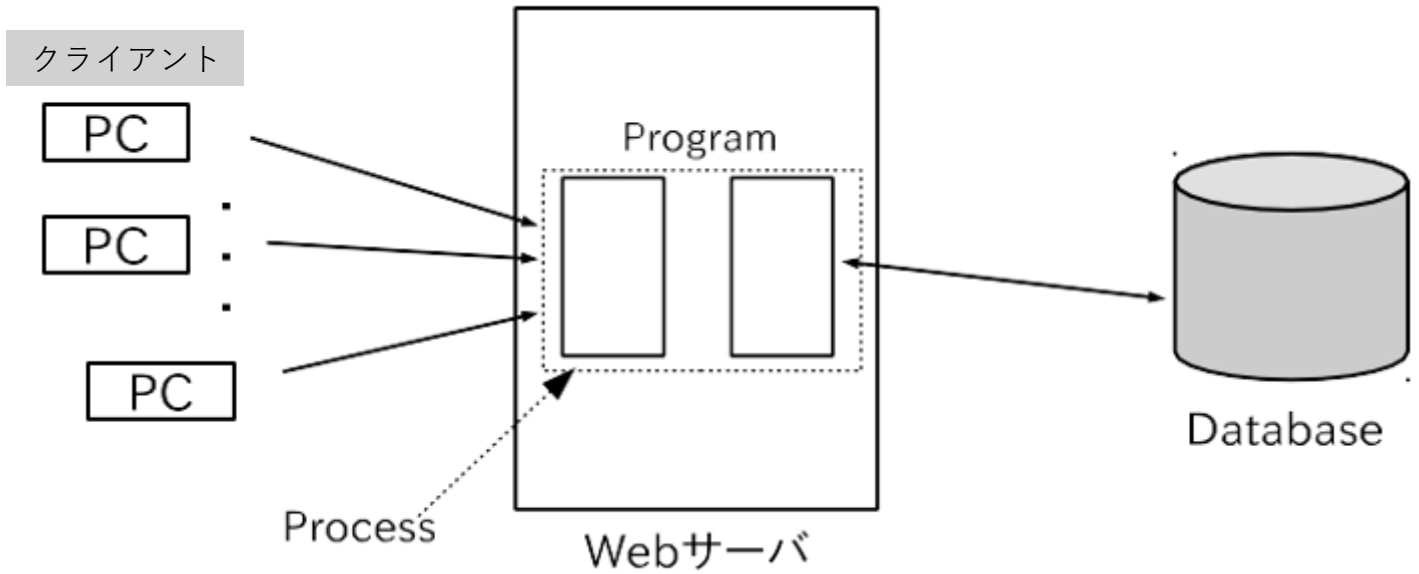
【復習】CPUの処理状況

- CPU 利用の効率化，待ち時間減少， 高速化のため，
  並列 (並行) 処理が多用される



並列処理なしのWebシステム

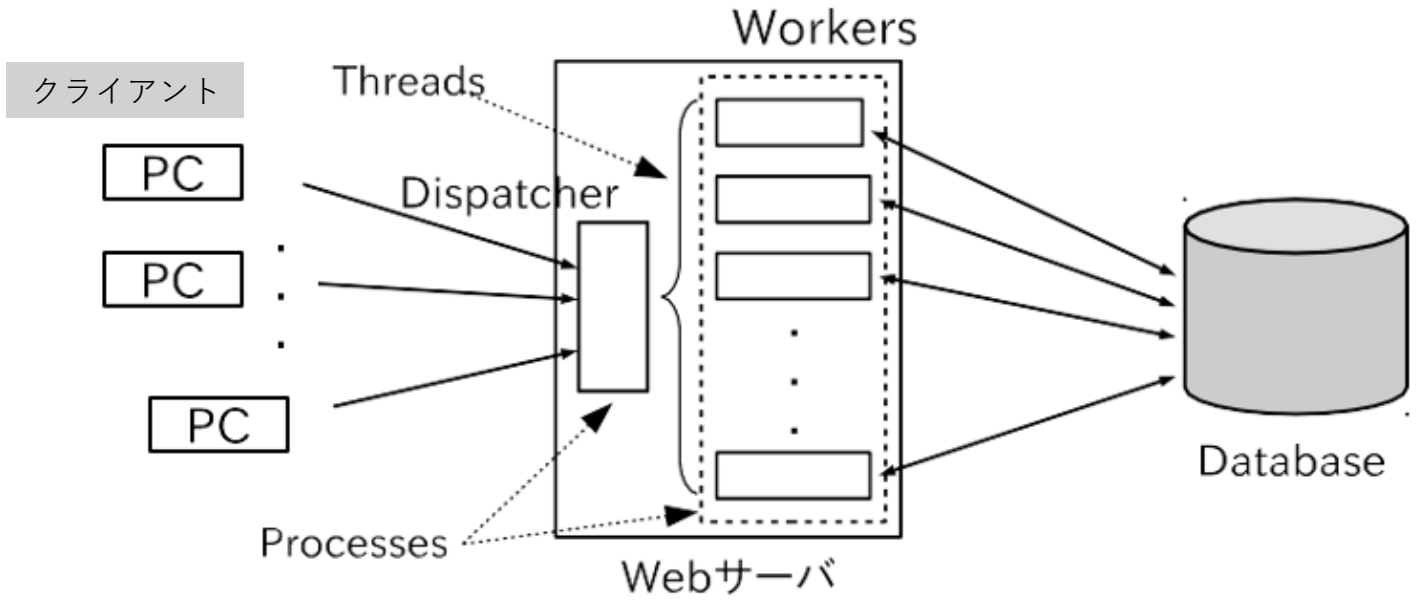
- 1 台のPC の一連の処理が終わるまで他のPC は待たされる
- 高速化には計算機処理 (I/O を含む) を高速化する必要がある



あるクライアントの一連の処理 [接続 → リクエスト処理 → 接続終了] が終了して初めて次のクライアントへ

並列処理を用いたWebシステム

- 各PC の処理は 各Worker で並列に実行されるので待たされにくい
- CPU が効率的に利用されるのでハードウェア自体の性能を過度に高速化する必要なし



クライアントとの接続 → 新スレッドを生成し [リクエスト処理] を投げる → 次のクライアントへ

並列処理と処理の高速化･効率化

- 計算機処理の高速化・効率化
  - 処理の並列化が鍵
  - PC上のOS/アプリケーションにおいて, (応答)性能の向上のためThread による並列化が多用(次スライド参照)
  - 例： pipeline, superscaler (忘れていたら思い出してください)
    高性能CPUではThread がハードウェアレベルで並列化
- スーパーコンピュータ
  - 超並列計算機 (三宅先生: 並列計算 参照)
- 「並列」計算機でなくても内部処理で並行処理は利用される

(参考) Windowsにおける各プロセスのThreads

- Powershell アプリを起動
  - 以下の powershell comandlet を起動

  - Get-process (cr)　.... 実行中のプロセス一覧を表示
  - Get-process | fl *　.... 実行中のプロセスの諸状況を表示
  - Get-process ap-name | fl * .... ap-name というアプリの状況を表示

  - この中で，Threads ... の部分を参照
- MacOSの場合
  - アクティビティモニタアプリを起動（実機未確認）
  - https://support.apple.com/ja-jp/guide/activity-monitor/actmntr1001/mac
  - すいません．Macの実機を持っていないため例示できません．
  - ターミナル上で，ps xM コマンド実行でも表示される(らしい)

【復習】プロセス関連

- **プロセス**:
  - 動作中のプログラム
  - Task (タスク) とも呼ばれる
  - CPU スケジューリングの対象となる基本単位
- **スレッド**:
  - １つのプロセスの中に複数の制御の流れを実現
  - アドレス空間を共有 (プロセス間通信不要)
  - CPU スケジューリングの対象となる単位
- **マルチプログラミング**:
  - 複数のプロセスの並行実行による CPU の利用効率向上
- **CPU スケジューリング**:
  - FCFS スケジューリング
  - SJF スケジューリング
  - 優先度スケジューリング
  - ラウンドロビンスケジューリング
  - 多重レベルスケジューリング

並行プロセスとは (3.1)

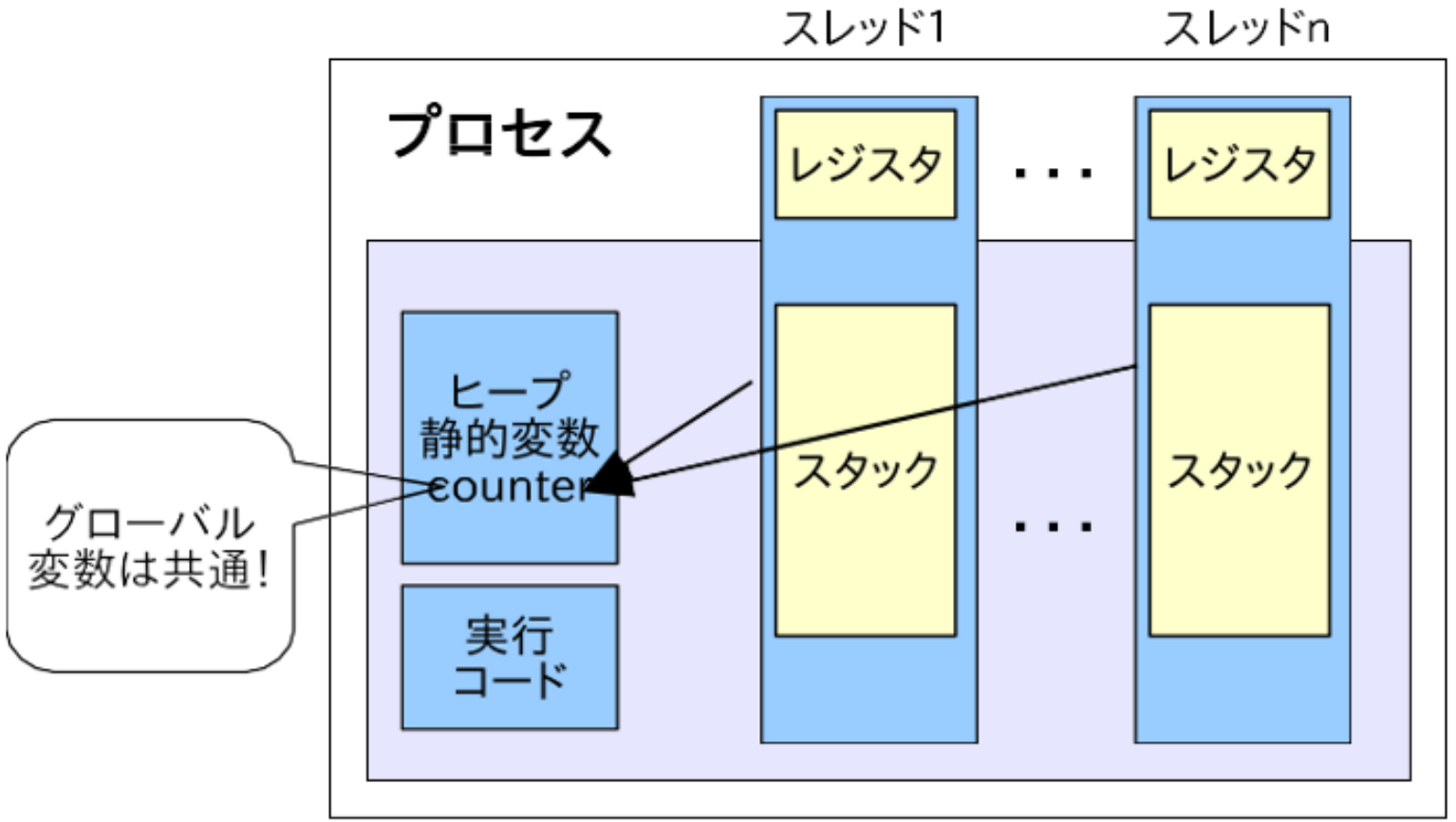
- 逐次プロセス (Sequential process):
  - 同時実行不可能
  - プロセス間に依存関係がある
- 並行プロセス (Concurrent process):
  - 2つ以上同時に実行可能
  - 「資源」を共有する場合，同期 (Synchronization) あるいは相互排除 (Mutual exclusion) が必要
  - 共通に操作するデータが存在しない → 素 (disjoint)
  - 共通に操作するデータが存在する → 交差 (overlapping)
    - 共同型逐次プロセス: 同時に実行可能であるが相互排除が必要

分散処理の記述には必須の概念

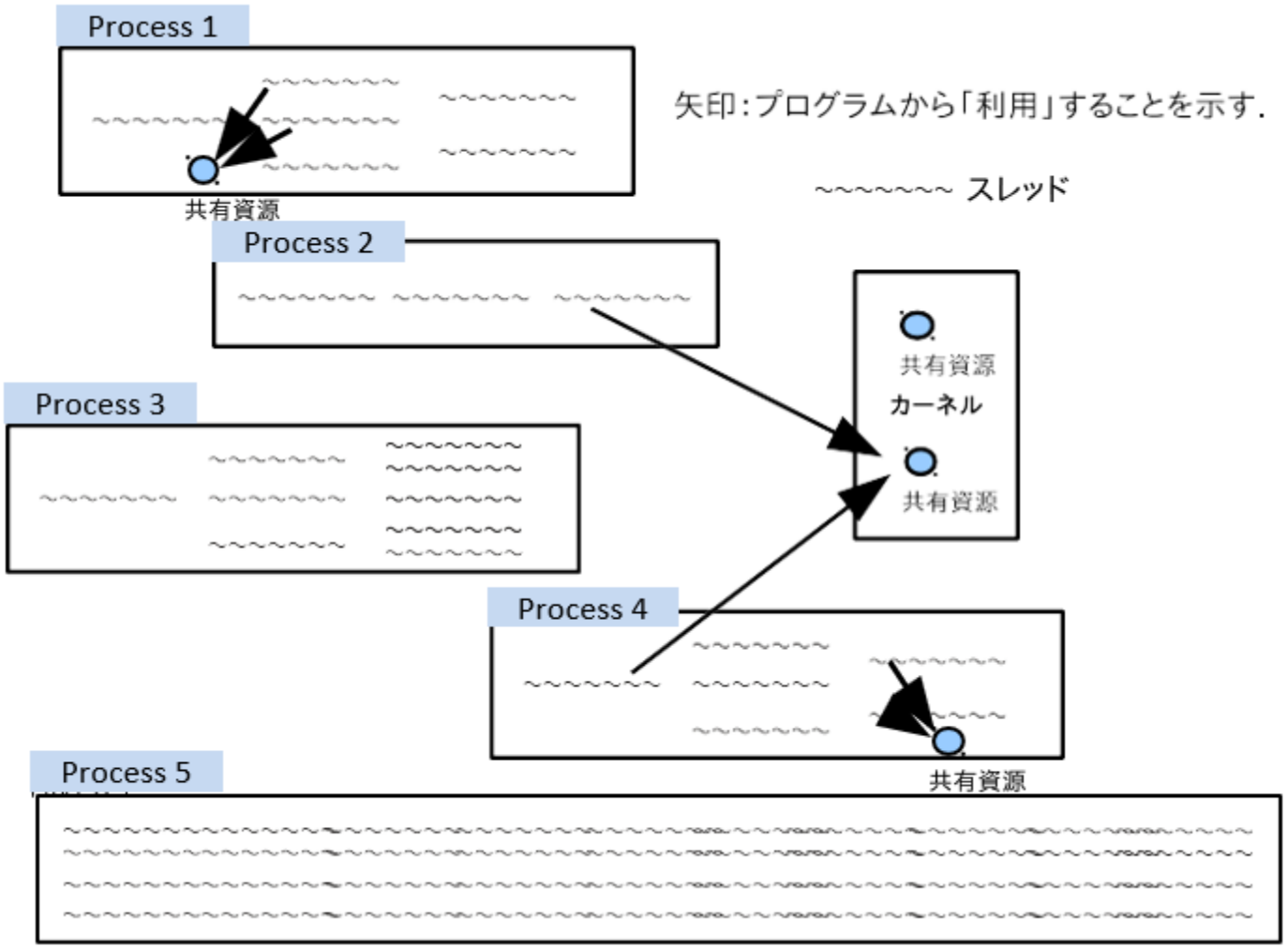
相互排除が必要な例

- 同一ファイル（メモリ）領域へのデータ書き込み
  - プロセスA, B が同時に異なるデータを同じファイル（メモリ）領域へ書こうとしたらどうなるか?
- 出力デバイスへの同時出力
  - プロセスA, B が同時にプリンタへデータを出力しようとしたらどうなるか？

相互排除が必要な例 (Thread: メモリ)



計算機全体では？


Process 1
共有資源
矢印：プログラムから「利用」することを示す．
スレッド
Process 2
共有資源
カーネル
共有資源
Process 3
Process 4
共有資源
Process 5

プロセスの同期と相互排除 (3.3)

- **共有資源 (shared resource)**:
  - プロセス間で共有されるプログラム，データなど
  - 一般に同時にアクセス可能なプロセス数が制限
    → 逐次的資源 (同時に 1 プロセス)
- **際どい領域 (critical section / region)**:
  - 各プロセスで，逐次的資源を使用する部分 (プログラム)
  - 臨界領域とも言う
- **相互排除 (排他制御)**:
  - あるプロセスが資源を利用している場合，資源が解放されるまで他のプロセスを待たせること
  - 目的: デッドロックやデータ破壊の回避
  - リソースが衝突する可能性が (僅かでも) あれば，相互排除を行う必要あり

プロセス間の同期と相互排除

- プロセス間における事象の同期 (連絡):
  - あるプロセスの処理が他のプロセスの処理に依存する場合に必要
    → 分散処理でたびたび使用される処理
  - 例 1: あるプロセスは他のあるプロセスの処理の結果を待つ
    (処理の結果が「連絡」される)
  - 例 2: 生産者・消費者問題において，倉庫が空の場合，消費者は，
    生産者が生産して倉庫に製品を入れるのを待つ
    (製品が倉庫に入ったことが「連絡」される)
- 多くの場合，相互排除と同期は組み合わせて利用

相互排除機構

- 割込み禁止
- Busy Waiting による相互排除
  - ロック変数
  - 同期基本命令
  - Dekker のアルゴリズム
- Sleep/Wake up による相互排除
  - セマフォ
  - モニタ

割込み禁止による相互排除

- プロセスの切り替えは割込みにより実行される
  - 割込みを禁止すれば割り込まれることはない
  - 際どい領域に入ったら最初に割込みを禁止命令を実行,
    出る直前に割込み許可命令を実行

- 問題点
  - 利用できる状況が限られる
  - BIOS，デバイスドライバ
  - 割込み禁止時間の綿密な設定が必要
  - 複雑な処理においては利用困難

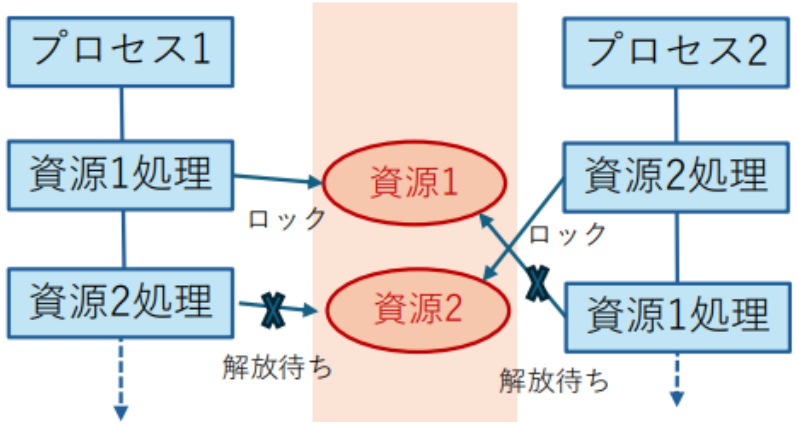
Peterson’s algorithm (Dekker のアルゴリズム) (3.3.1)

```
Program Dekker
 var turn integer;
 c1, c2: Boolean;
 Procedure P1; /*  プロセスP1 */
 begin
   while true do
      begin
        c1 := true; /* 臨界領域に入る宣言 */
        while c2 do /* c2が偽ならP1優先なので臨界領域CS1へ */
          if turn = 2 then /* P2が臨界領域に入っているか */
             begin
               c1 := false
               while turn = 2 do /* P2が臨界領域から出るまで待つ */
                c1 := true;
              end;

        CS1;  /* プロセス1の臨界領域 */
        turn := 2; / * P2に優先権を渡す (今度はPS2が臨界領域に入る番) */
        c1:= false;
        NCS1 /* 臨界領域以外の処理 */
      end
end;
/* プロセス2については省略 */
```

```
begin
   turn := 1;
   c1 := false;
   c2 := false;
   cobegin
      P1; P2 /* P1,P2を平行に実行 */
   coend
end.
```

・c1とc2で相互実行が起こりえないことを保証
・turnでデッドロックが発生しないことを保証



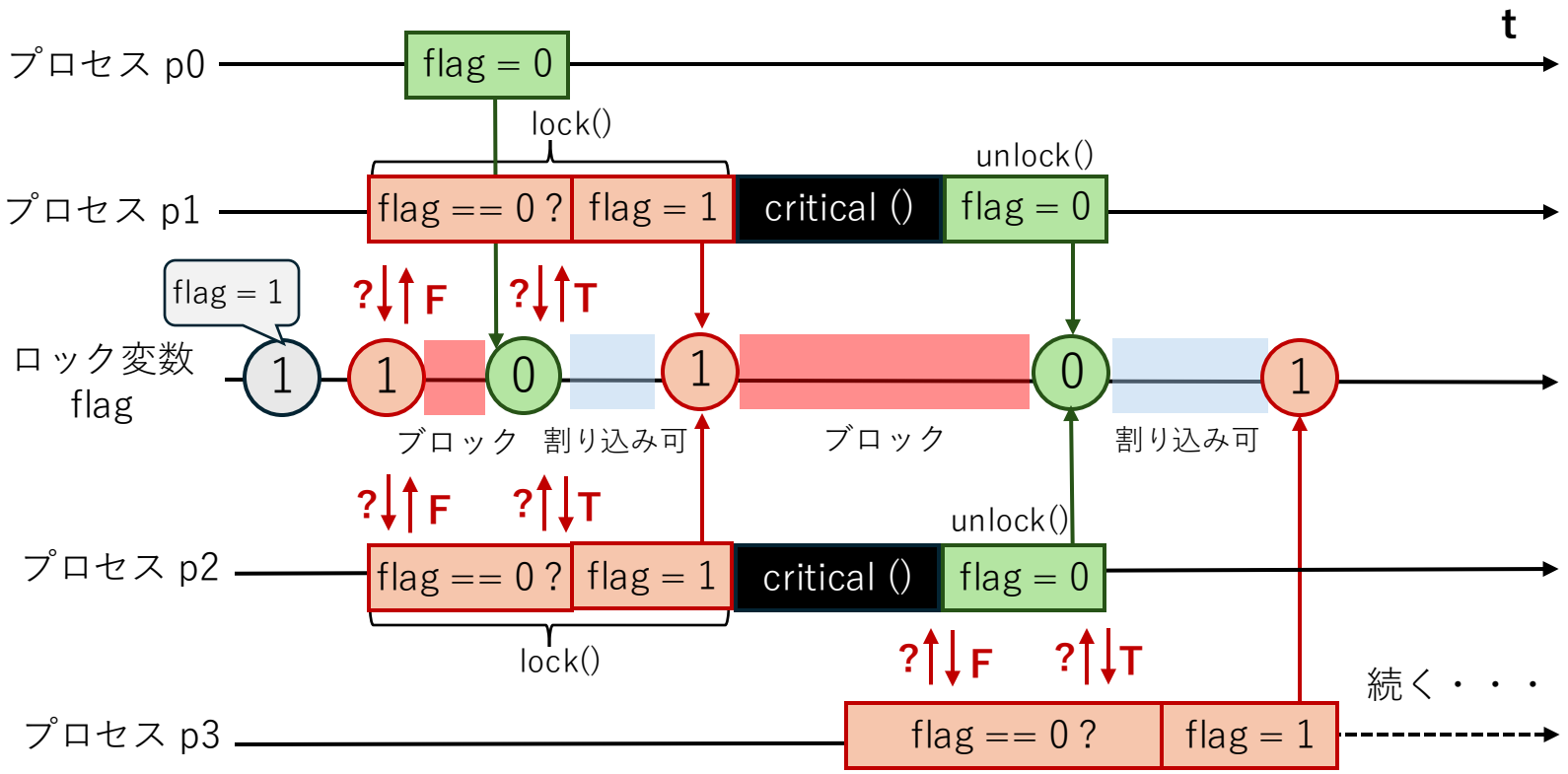
Perterson's アルゴリズム(Dekkerのアルゴリズム)(3.3.1)

- 1981年にG.L. PetersonがオリジナルのDekkerのアルゴリズムより簡単なアルゴリズムを発見
- 主記憶への格納操作が不可分であることを利用した,
  最も低水準の相互排除操作
  - P2 の番か? → P2 も臨界領域か？ → 待つ
  - P2 の番かつ P2 も臨界領域か？ → 待つ
- 実用的ではない → 相互排除の原理を明らかにしたという意義
  - 理由：相互排除の対象となるプロセスが事前に既知である必要がある!
  - 現実の「相互排除」には場合に応じて後に説明する手法が用いられる.
- N 個のプロセスの場合に適用するにはどうすればよいか考えよ

ロック変数による相互排除

- プロセス間でロック変数 (lock) を共有
- lock == 0 の場合:
  - (際どい領域にプロセスはない)
  lock = 1 と設定して際どい領域に入る
  - 際どい領域から出る時に lock = 0 とする
- lock == 1 の場合:
  - (際どい領域にプロセスが入っているとして) lock=0 となるのを待つ
- 問題点: 安全なアルゴリズムではない
  - 複数の機械語命令で実行されるため，命令間で割込みが発生する可能性あり
  - あるプロセスがlock == 0 を確認して，lock=1 とする間に他のプロセスが割り込んだら????
  - (上記の問題を避けるためには，不可分な) 同期基本命令を利用

ビジーウェイト1: ロック変数による相互排除の例



- プロセス間でロック変数 を共有
- 変数flagの値を確認した後に割り込みが起こり，不整合が発生する恐れがある
- 割り込まれることなく更新する機能が必要
  → 不可分な同期基本命令を使用

2つのプロセスが同時に際どい領域に入っている

```
lock();
critical(); // ③
unlock();
```

```
lock(){
  while(flag == 0); // ①
  flag = 1; // ②
}
```

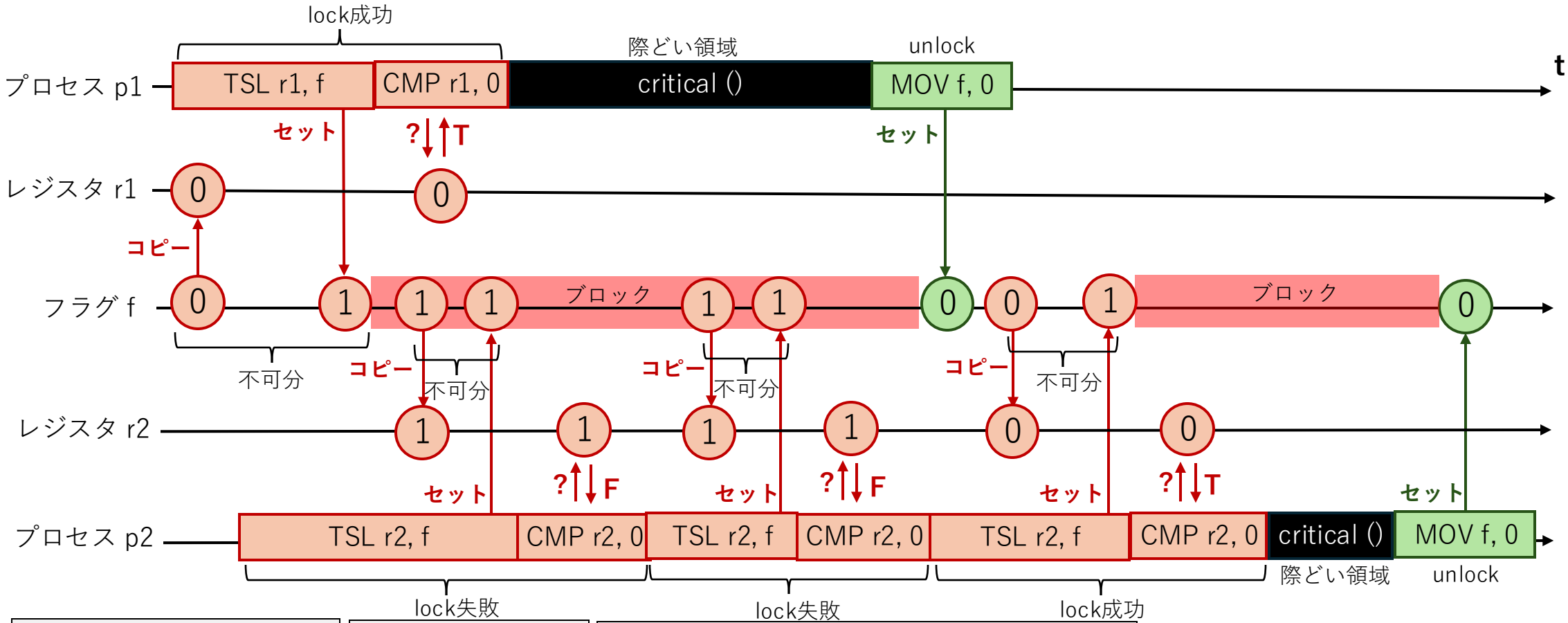
```
unlock(){
  flag = 0; // ④
}
```

① flagの値が1の間はループ
② flag = 0になったらflagの値を1にセットして きわどい領域に入る
③ きわどい領域の処理（関数critical()）を実行
④ flagの値を0にしてきわどい領域を出る

TS命令 (3.3.2)

- TS 命令 (test and set):
  - 共有変数の値の読み出しとテスト
  - 共有変数への新しい値の設定
    - 上記の処理が「不可分」に実行される(Atomic とも呼ばれる)
  - 単一の機械語命令
- 特徴
  - 実行中に割込みが発生しない
  - Busy-wait が発生する
    - 常に繰り返しロック変数 (spin lock) をテスト
      - CPU が無駄使いされる
      - システム性能の低下につながる

ビジーウェイト2: TS命令による相互排除の例




```
lock:
  TSL register, flag  // ①
  CMP register, 0     // ②
  JNZ lock            // ③
  RET                 // ④
```

```
unlock:
  MOV flag  // ⑤
  RET // ⑥
```

① flagの値をレジスタにコピーしてflag値を1に
② レジスタにコピーされたflag値は0か?
③ 0でなければlockの冒頭へ
④ 0なら呼び出し元に戻り際どい領域へ
⑤ flag値を0に
⑥ 呼び出し元に戻る

チェック前にロックして割り込みを防止している

但しループ中はビジー(プロセッサを使用)

Busy-wait と Sleep and Wakeup

- Busy-wait
  - 常に繰り返しロック変数 (spin lock) をテスト (tight loop に陥る)
  - CPU が無駄使いされる
  - システム性能の低下につながる
- Sleep and Wakeup
  - sleep システムコールを利用
    - sleep している場合，CPU は他に割り当てられる
  - awake システムコール
    - sleep している場合，CPU の割当を開始

セマフォによる相互排除 (3.3.3)

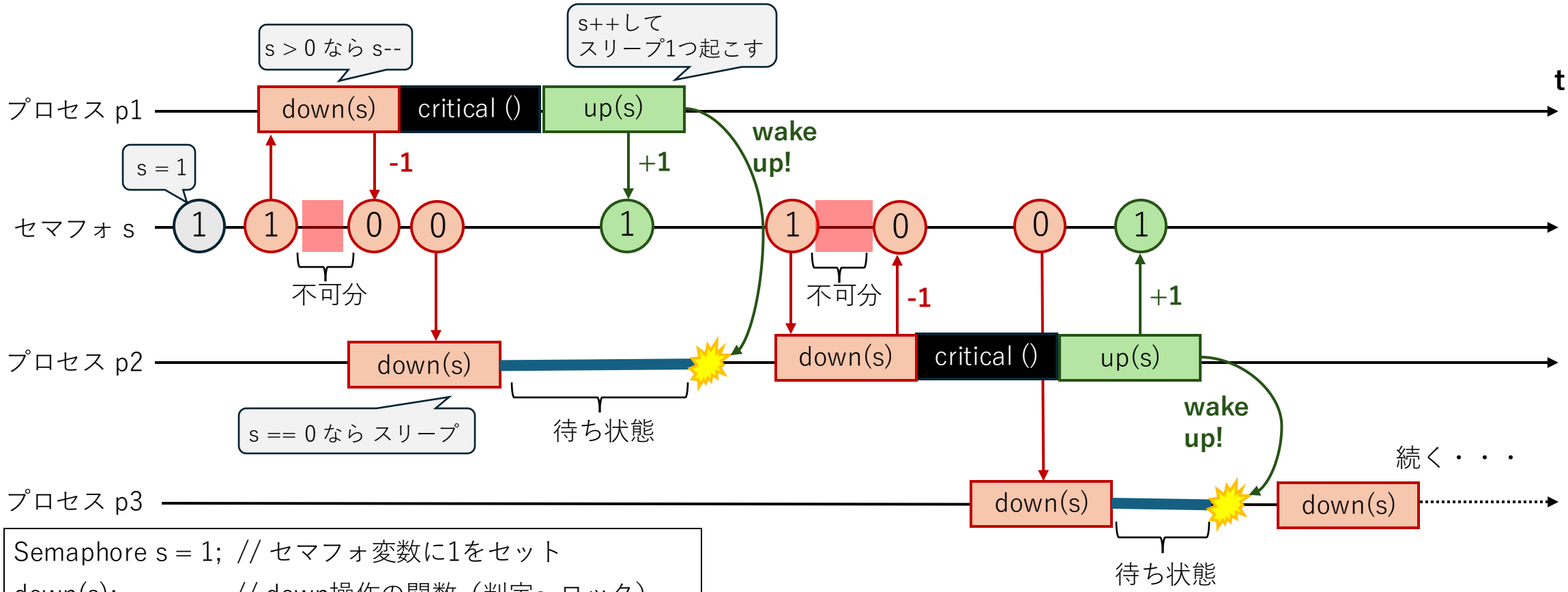
- 腕木信号機がモデル (1968 年 Dijkstra)
- “semaphore” 変数を導入（同期用）
- Down, Up (P, V) の 2 種類の操作 (不可分な命令)
  - Down (P) 操作 (共有資源が使用可能になるまで待つ)
    - s > 0 ならば s := s – 1 → 次の処理を継続
    - s == 0 ならば sleep (待ち行列に入る)
  - Up (V) 操作 (共有資源を解放)
    - s := s + 1
    - 待ち行列から1つ選び down 操作の終了 (sleep解除) を許可



Semaphoreの典型的な利用例

```
Semaphore S = 初期値; /* 1 or 2（1の場合同時1, 2の場合同時2*/
Process 1, 2, 3, 4
/* ソースは共通で記述. 同じである必要はない*/
…
down(S);                    /*Down (P) 操作*/
Critical_region();    /* Critical Region にあるコード */
up(S);                      /* Up (V) 操作 */
…
```

2進 (binary) セマフォによる相互排除の例



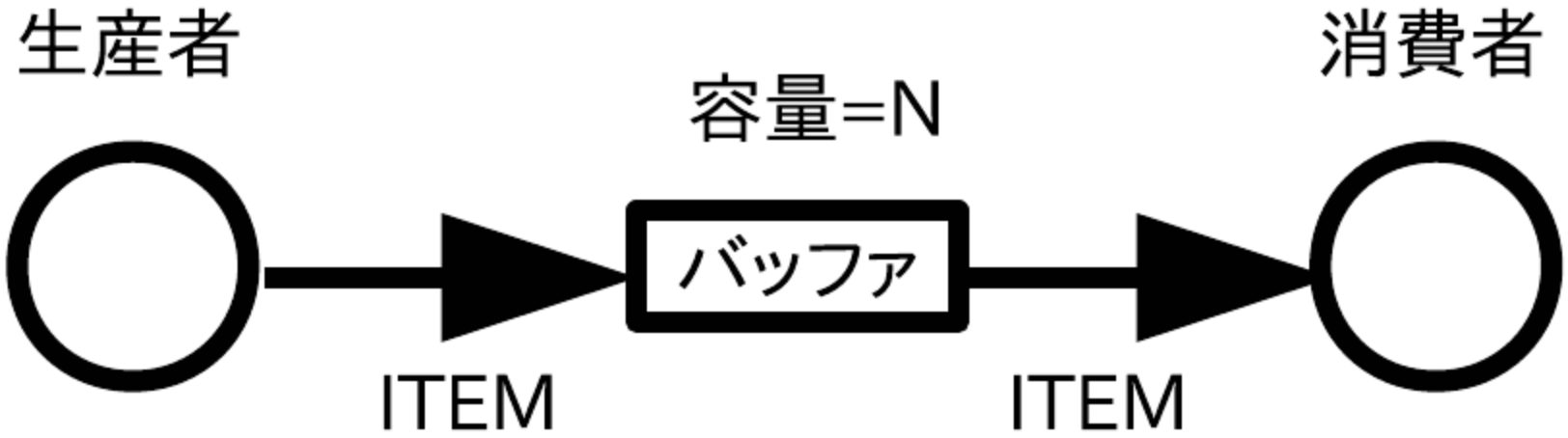

```
Semaphore s = 1;  // セマフォ変数に1をセット
down(s);          // down操作の関数（判定～ロック）
critical();       // きわどい領域の関数
up(s);            // up操作の関数（アンロック）
```

初期値が2の場合 (同時に2プロセスまで臨界領域に入れる) はどうなるか考えてみよ

セマフォの実装

- down /up 操作は，（不可分な）システムコールとして実現
  - セマフォの値のチェック・更新
  - プロセスのスリープ
- 軽微な処理なので，　割り込み禁止も問題なし

同期･連絡を伴う事例



生産者：　製品を生産し，倉庫（バッファ）に搬入する
消費者：　製品を購入し，倉庫（バッファ）から運び出す

倉庫：　　容量に制限(N)が存在.

生産者：　倉庫が満杯の場合は，搬入（生産）を待つ
消費者：　倉庫が空(在庫がない)場合は，搬入されるまで待つ

連絡：　生産者，倉庫が一杯でなくなったこと，消費者：倉庫が空でなくなったこと

同期・連絡を伴うSemaphore 利用例 (プログラム概要)

```
#define N 100
typedef int semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

void producer(void){
   int item;
   while(TRUE){
      item = produce_item();
      down(&empty);
      down(&mutex);
      insert_item(item);
      up(&mutex);
      up(&full);
   }
}
```

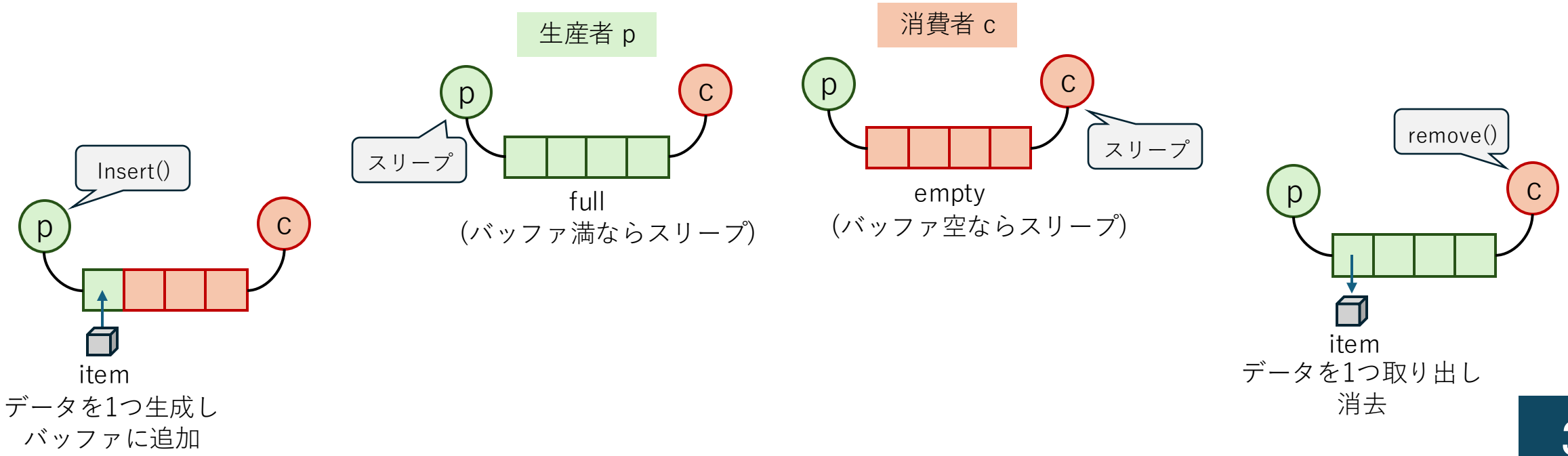
```
void consumer(void){
   int item;
   while(TRUE){
      down(&full);
      down(&mutex);
      item = remove_item();
      up(&mutex);
      up(&empty);
      consume_item(item);
   }
}
```

【参考】Semaphoreの例 (C言語)

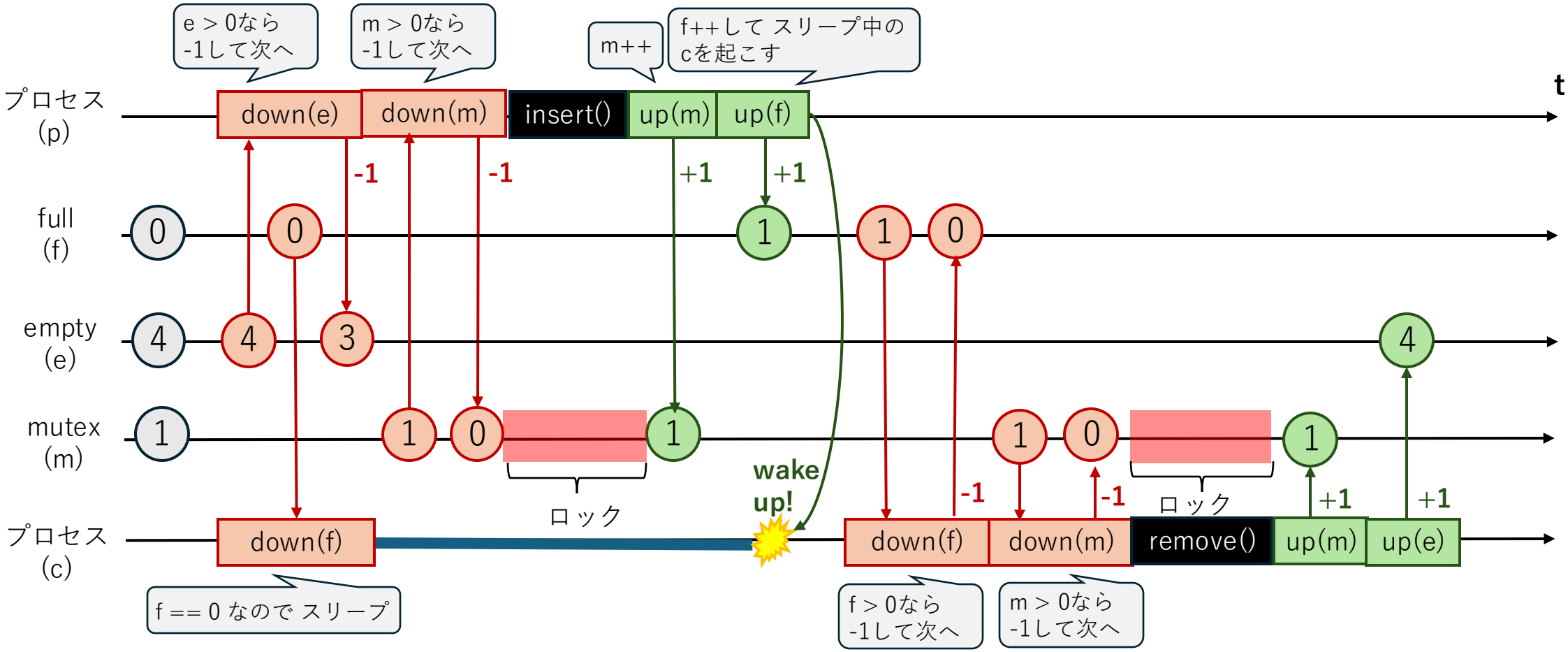
- 別プロセス間での相互排除:
  - semabinit.c , sema.c, semb.c をコンパイル・リンク
  - 最初に semabinit を実行
  - semaphore を確保 (名前 1492)
  - 2つterminal window を開いてそれぞれのwindowsで，sema/semb を実行
- Thread による相互排除の利用:
  - Producer-Consumer 問題
  - gcc -o p-c p-c.c -lpthread　でコンパイル・リンク
  - ./p-c で実行
  - このプログラムは前スライドのアルゴリズムとは異なっている．適当なバッファ容量 N を定義して, 前スライド のアルゴリズムを実現してみよ.

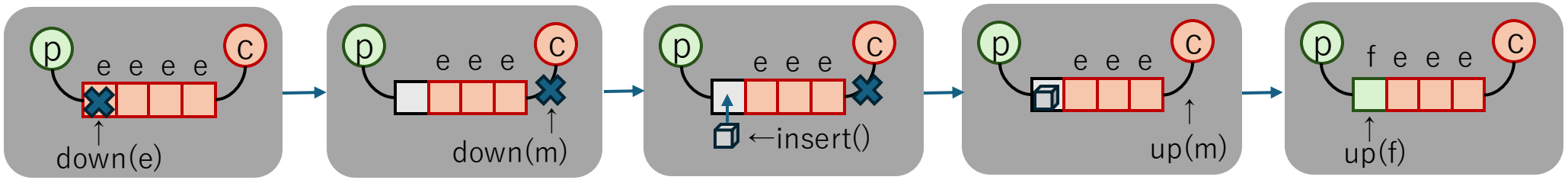
計数 (counting) セマフォによる読み書き同期の例

- **プロデューサ・コンシューマ問題:**
  - 複数プロセス間で通信を行い，データ交換する際の典型的な例
  - 固定長のバッファを共有メモリとして持つ2種類のプロセス「生産者 (producer)」と「消費者 (consumer)」が登場
  - カウンティングセマフォ (full：満 / empty：空 / condition：ロック)を用いる

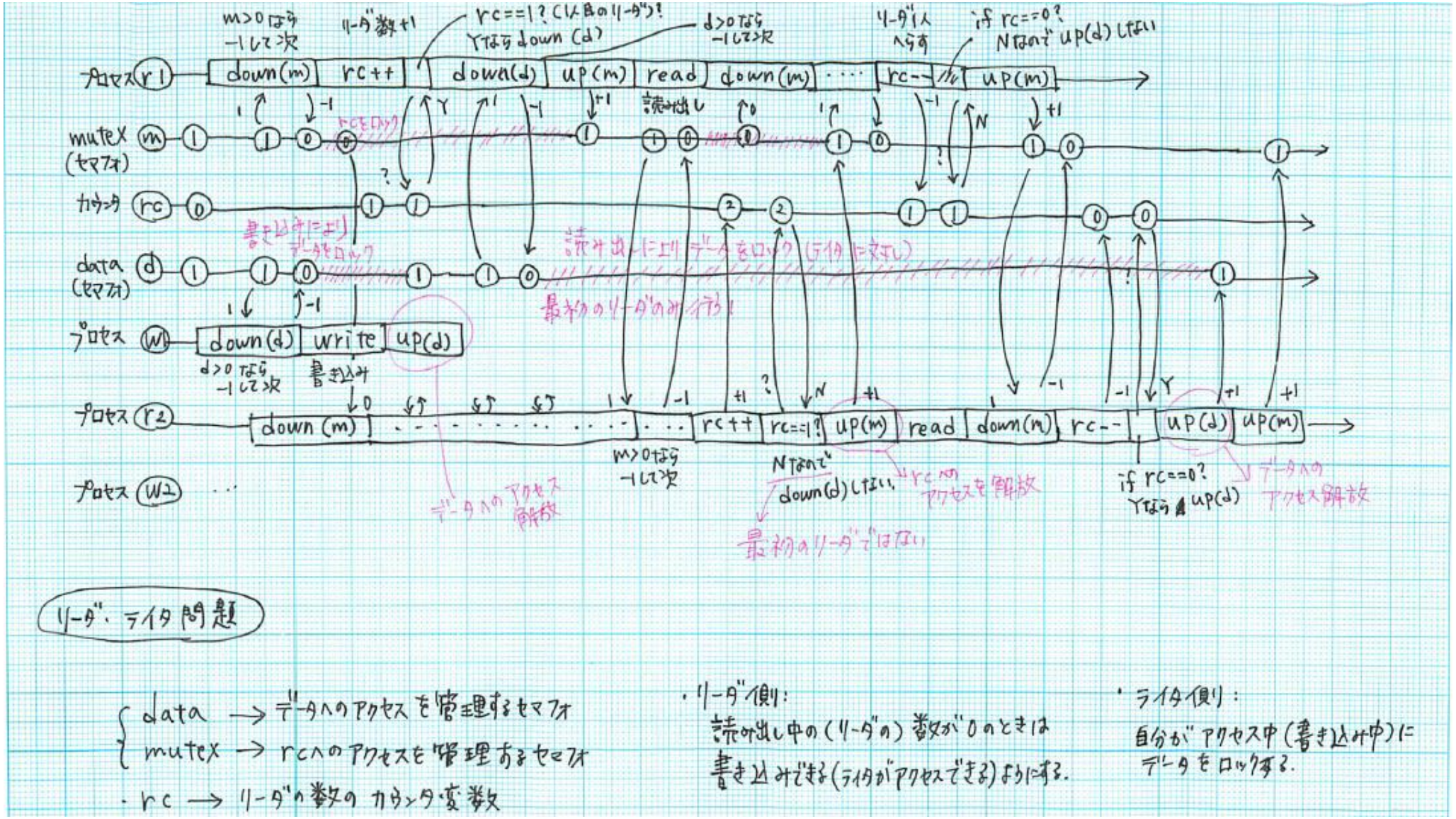


計数 (counting) セマフォによる読み書き同期の例





【参考】リーダ・ライタ問題



リーダ・ライタ問題

- data → データへのアクセスを管理するセマフォ
- mutex → rcへのアクセスを管理するセマフォ
- rc → リーダの数のカウンタ変数

・リーダ側:
読み出し中の(リーダの)数が0のときは書き込みできる(ライタがアクセスできる)ようにする.

・ライタ側:
自分がアクセス中(書き込み中)にデータをロックする.

セマフォの欠点

- down操作，up 操作の指定順所を誤り易い
  - if 文などの条件文で意図せずにup/down 操作を飛び越す
  - down 操作の実行時にセマフォ値が0となっていると，
    待つこと以外の操作はできない
- セマフォを利用しないプロセスにもセマフォを参照可能
- 同期・連絡のためにはテクニックが必要でありややわかりにくい

モニタの概要

- モニタ (monitor):
  - Hoare and B. Hansen による同期のための高位プリミティブ (1974)
  - 共有資源とそれに対する操作，初期化のためのコードを一体化
  - プロセスは，モニタの手続きを呼べるが，変数に直接アクセスできない
- プログラマから見ると
  - １つのプロセスにしか実行できないよう定義された関数
  - プログラマは際どい領域の関数をモニタとして定義
  - 関数そのものの実行がロックとアンロックの役割 → アンロック忘れがない
- モニタの性質
  - モニタ内では一度には１つのプロセスのみがアクティブ
  - 間違いがプロセス中に局在化できる

モニタの操作

- 同期機構 (必要な相互排除機能を含む)
- 条件変数 (condition variable)
  - wait 命令 条件変数が原因で待つ
  - signal 命令 待っているプロセスを起動
- 相互排除条件
  - signal 命令実行後はすぐモニタを出なければならない.
- 条件変数はカウンタではない
  - wait 命令は, signal 命令の前になければならない.

Monitor の利用の例(概要)

```
monitor ProducerConsumer
condition full, empty;
integer count;

proceduer insert(item: integer);
begin
    if count = N then wait(full);
    insert_item(item);
    count := count + 1;
    if count = 1 then signal(empty);
end;

function remove: integer;
begin
    if count = 0 then wait(empty);
    remove = remove_item;
    count := count - 1;
    if count = N-1 then signal(full)
end;
```

```
procedure producer;
begin
    while true do
    begin
        item = produce_item;
        ProducerConsumer.insert(item)
    end
end;

procedure consumer;
begin
    while true do
    begin
        item = ProducerConsumer.remove;
        consume_item(item)
    end
end;
```

同期・連絡が絡む並列プログラムの記述に向いている